Building Accessible Android Applications

A Case Study on an Accessible Podcast & Audiobook App
built with Jetpack Compose and Kotlin

A technical report on semantic accessibility, screen-reader support, and accessibility testing in modern Android UI development.

Group of 4

Phát • Phong • Khương • Tính

Course: Mobile Application Development • Group of 4

July 25, 2026

Contents

1 Introduction

1.1 Motivation

Roughly one in six people worldwide lives with a significant disability. For many of them a smartphone is not a convenience but the primary way they read, communicate, navigate, and work. *Accessibility* is the practice of designing hardware and software so that people with temporary or permanent physical, sensory, or cognitive differences have an equal opportunity to use them. When an app ignores accessibility, it does not merely inconvenience these users — it excludes them from a part of modern life. Building inclusively is therefore both an act of basic fairness and, increasingly, a legal obligation.

This report documents how our group re-engineered a common consumer application — a **podcast and audiobook player** — to be usable by people who rely on assistive technology such as the TalkBack screen reader, Switch Access, and large display fonts. We deliberately chose an audio-first product: listening to spoken content is one of the few digital experiences that a blind user can enjoy on completely equal terms with a sighted user, provided the surrounding interface is accessible.

1.2 Project goal and scope

Our goal was to take a typical, "accessibility-blind" podcast UI and *improve* it into an accessible one, then explain and justify every technique we applied. The deliverable is a front-end-only Jetpack Compose application (all data is mocked behind a repository interface, with no real backend) plus this report, which is organised around four contributions:

- **Background** — the legal and technical foundations of Android accessibility.
- **Core semantics** — labelling, roles, state, touch targets, and font scaling.
- **Advanced semantics** — merging, traversal order, and custom screen-reader actions.
- **Testing & evaluation** — how we verify the result and honestly assess trade-offs.

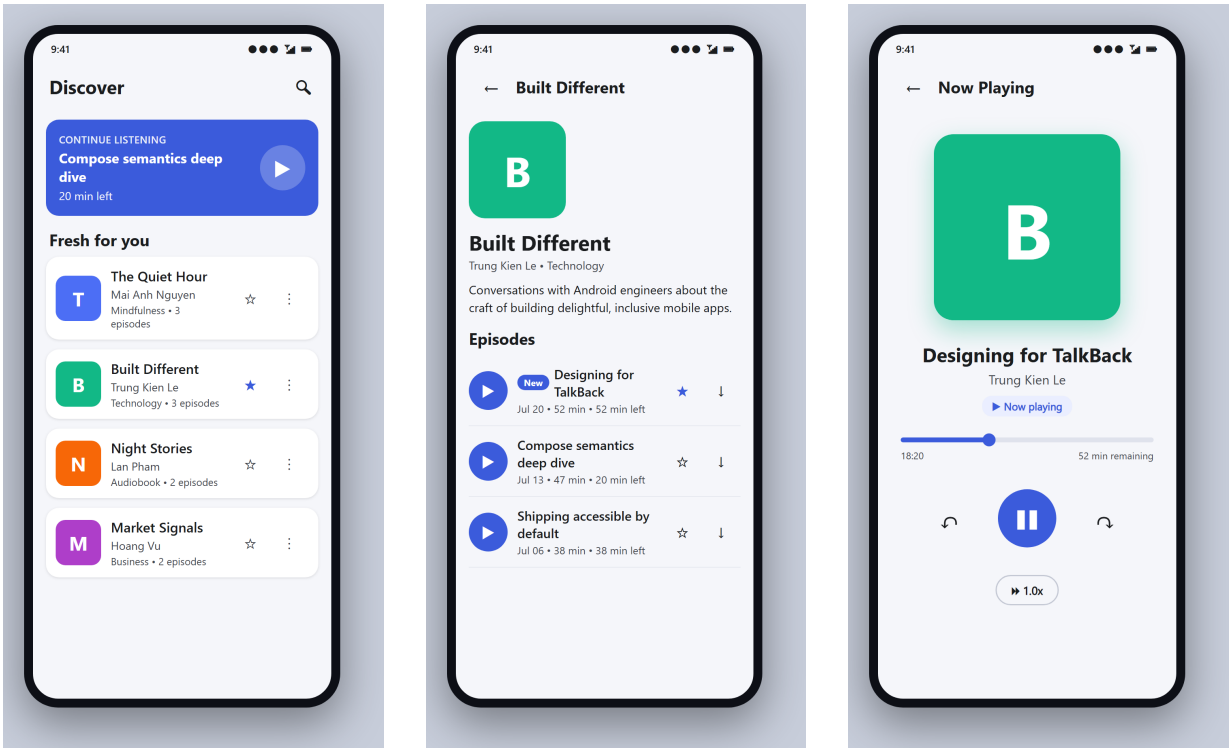
1.3 What "done" looks like

Throughout the project we held ourselves to a simple bar: *a person who never looks at the screen should be able to open the app, browse shows, pick an episode, and control playback using TalkBack alone*. Figure 1 shows the three-screen user flow that a sighted user sees; the rest of the report explains what a screen-reader user hears and does at each step.

2 Background

2.1 Accessibility as a legal requirement

Accessibility has moved from "nice to have" to a statutory duty in many markets. The **European Accessibility Act (EAA)** argues that a common accessibility standard benefits everyone: disabled users gain fair, unified accommodation and real competition between providers, while companies gain a larger customer base and a single ruleset to build against. All EU member states incorporated the EAA into national law from June 2022, leaving the exact mechanism (directive, act, or regulation) to each country; the shared principle is that products offered in the EU must satisfy accessibility requirements.



(a) Discover **(b)** Podcast detail **(c)** Now Playing

Figure 1: The three screens of the case-study app. The user flow is **Discover** → **Podcast detail** → **Now Playing**, with a "Continue listening" shortcut on the home screen.

The technical yardstick behind most of this legislation is the **Web Content Accessibility Guidelines (WCAG 2.2)**. Although WCAG is written for the web, its success criteria are applied to native software through *WCAG2ICT* and platform-specific mobile guidance. Because a podcast app is not a web page, we treat WCAG as *technology-neutral guidance* and map each criterion onto its Android equivalent rather than claiming literal "compliance".

2.2 The POUR principles

WCAG organises accessibility around four principles, remembered by the acronym **POUR** (Figure 2). They are the lens through which we evaluated every screen of our app.

Perceivable
Information must be presentable in ways users can sense — text alternatives, captions, sufficient contrast.

Operable
Every action must be reachable without a mouse — keyboard, switch, and screen-reader navigation.

Understandable
Content and controls must be predictable, clearly labelled, and give helpful error feedback.

Robust
The UI must expose reliable semantics so current and future assistive tech can interpret it.

Figure 2: The four POUR principles that underpin WCAG and this project.

For a media player these translate concretely: **Perceivable** means labelling every transport icon and keeping duration text high-contrast; **Operable** means play, seek, and bookmark must all work from TalkBack; **Understandable** means the play button announces whether it is playing or paused; **Robust** means we expose a correct *semantics tree* rather than relying on pixels.

2.3 How Android exposes the UI to assistive technology

The single most important idea for an Android developer is that the screen a sighted user sees and the structure a screen reader traverses are *two different trees* (Figure 3).

- The **Composition / layout tree** is the visual render hierarchy. Its nodes are boxes, text, and images, ordered by where they are drawn on screen.
- The **Semantics tree** runs in parallel and describes *meaning*: each node carries a name/label, a role, a state, and the actions it supports. This is the only thing assistive services can read.

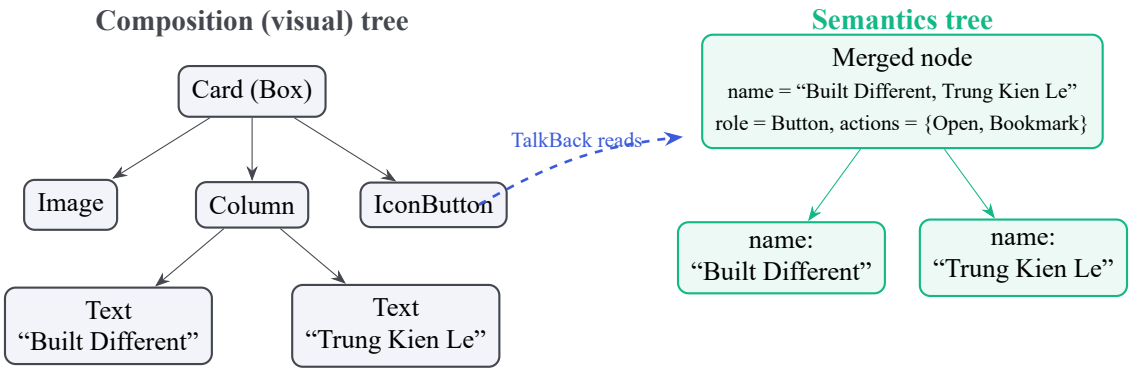


Figure 3: The visual tree (left) is what is drawn; the semantics tree (right) is what TalkBack reads. The decorative image is dropped and the two texts are *merged* into one meaningful node — techniques covered in Sections 4 and 5.

Assistive services such as TalkBack (screen reader) and Switch Access run as background system services. They subscribe to the *merged* semantics tree, convert each node into speech or a scan target, and translate user gestures back into actions. A screen can therefore look perfect yet be unusable if its semantics tree is missing, wrong, or badly ordered. Everything that follows is about shaping that tree correctly in Jetpack Compose.

2.4 Jetpack Compose in one paragraph

Jetpack Compose is Android’s modern declarative UI toolkit (conceptually similar to React). UI is described by *composable functions*; state changes trigger recomposition. Crucially, Compose generates the semantics tree from `Modifier.semantics` blocks and from the built-in semantics of Material components, which gives us fine-grained, code-level control over exactly what assistive technology perceives.

3 The Case-Study App: *Accessible Podcast*

3.1 Overview

To ground the techniques in something concrete, we built *Accessible Podcast*, a small but complete Jetpack Compose application. It is intentionally **front-end only**: there is no network or database. All content is served by an in-memory mock that implements a repository interface, so the UI code is identical to what it would be against a real backend, but the project runs anywhere with no configuration.

3.2 User flow

The app implements the classic three-step listening flow, plus a resume shortcut (Figure 4).

Figure 4: User flow. Each transition is reachable by TalkBack double-tap and by Switch Access.

3.3 Architecture

The code is organised into a thin data layer and a Compose UI layer (Figure 5). The UI depends only on the `PodcastRepository` interface, never on the mock, which is the “interface / mock” boundary that stands in for a backend.

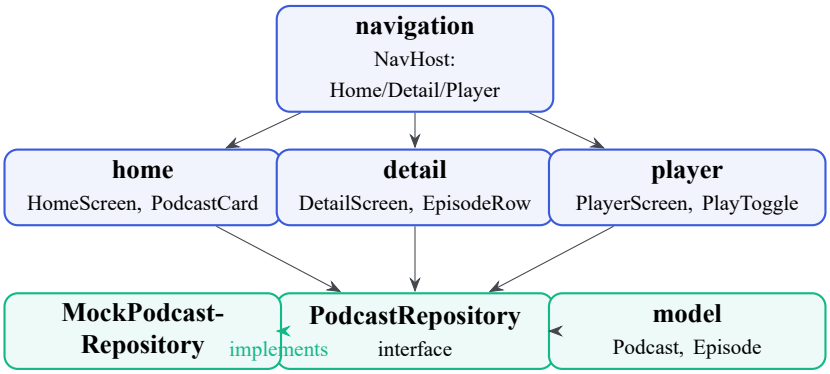


Figure 5: Package architecture. The Compose UI layer (blue) talks to a single repository interface; the mock (green) is the only implementation and can be swapped for a real one with no UI change.

3.4 The mock data layer

The repository exposes an observable catalogue as a `StateFlow`. Toggling a bookmark mutates the flow, so every screen recomposes exactly as it would against a reactive backend — which also lets us verify that state changes are announced to TalkBack.

```
interface PodcastRepository {
    val podcasts: StateFlow<List<Podcast>>   // observable catalogue
    fun getEpisode(id: String): Episode?
    fun toggleBookmark(episodeId: String)     // mutates state -> recomposition
    fun toggleDownload(episodeId: String)
}
```

Listing 1: The repository interface the whole UI depends on.

3.5 Where each technique lives

Table 1 maps every accessibility technique discussed in this report to the exact file in the project, so the demo and the prose stay in lock-step.

Report section	Technique	File
Core (§4)	`contentDescription`, decorative `null`	`ui/components/CoverArt.kt`
Core	`Role.Button` + `stateDescription`	`ui/player/PlayToggle.kt`
Core	48 dp touch targets	`ui/home/PodcastCard.kt`
Core	`sp` typography, font scaling	`ui/theme/Type.kt`
Core	heading navigation	`ui/home/HomeScreen.kt`
Advanced (§5)	`mergeDescendants`	`PodcastCard.kt`, `EpisodeRow.kt`
Advanced	`clearAndSetSemantics` + `customActions`	`EpisodeRow.kt`
Advanced	`hideFromAccessibility`	`CoverArt.kt`
Advanced	`isTraversalGroup` + `traversalIndex`	`HomeScreen.kt`
Background	`liveRegion` announcements	`ui/player/PlayerScreen.kt`
Testing (§6)	Compose assertions, ATF	`androidTest/AccessibilityTest.kt`

Table 1: Technique-to-file map. Every claim in the report is backed by runnable code.

4 Technical Approach I — Core Semantics

This section covers the foundational modifiers that make an individual component accessible: labelling, roles, state descriptions, touch-target sizing, and scalable typography. These are the techniques that fix the most common and most damaging defects a screen-reader user encounters.

4.1 Content descriptions: naming visual elements

Screen readers cannot "see" an icon; they can only read its text alternative. In Compose this is the `contentDescription` parameter. Three rules govern good labels:

- **Succinct & action-oriented.** Avoid "Image of" / "Button to" — the role is announced separately.
- **Localized.** Always use `stringResource()` so labels translate with the device language.
- **Hide the decorative.** Purely decorative graphics must be removed from the tree so they do not add noise.

```
// Meaningful control — announced as "Search podcasts, button"
Icon(
    imageVector = Icons.Filled.Search,
    contentDescription = stringResource(R.string.cd_search)
)

// Decorative cover tile — next to a text title, so it adds nothing for TalkBack
Box(
    modifier = Modifier.size(56.dp).background(color)
        .semantics { hideFromAccessibility() }   // Compose 1.8+; still visible to UI tests
)
```

Listing 2: Meaningful icons get a localized label; decorative art is hidden.

Real apps apply exactly this split: Instagram gives its action buttons descriptive labels ("Like", "Comment") but passes `null` to the decorative gradients around stories, so screen-reader users move between real content quickly.

4.2 Roles and state descriptions on custom controls

Material components carry their own semantics, but a custom control built from a bare `Box` tells assistive tech nothing. We must declare two things explicitly: a **role** (so TalkBack appends "Button"/"Switch") and, for anything toggleable, a **state description** (the dynamic "Playing"/"Paused"). Our circular player button, taken directly from the app, does both:

```
Composable
fun PlayToggle(isPlaying: Boolean, onToggle: () -> Unit) {
    val name = stringResource(if (isPlaying) R.string.cd_pause else R.string.cd_play)
    val state = stringResource(if (isPlaying) R.string.state_playing else R.string.state_paused)
    Box(
        modifier = Modifier.size(72.dp).clip(CircleShape).background(primary)
            .clickable(onClick = onToggle, role = Role.Button)
            .semantics {
                contentDescription = name       // "Play" / "Pause"
                stateDescription = state         // "Playing" / "Paused"
                role = Role.Button
            },
        contentAlignment = Alignment.Center
    ) {
        Icon(imageVector = icon, contentDescription = null) // handled by the Box
    }
}
```

Listing 3: `PlayToggle.kt` — a custom control with an explicit role and a dynamic state.

TalkBack now announces *"Pause, button, Playing"* — name, role, and state — which is exactly how Spotify's custom playback control behaves.

4.3 Touch-target sizing

Users with motor impairments, or anyone tapping on a moving bus, need a target they can hit reliably. Material Design mandates a minimum interactive size of **48 dp** × **48 dp** (WCAG 2.5.5 asks for 44 px). The visual icon can stay small; only the *touchable area* must grow. Compose does this automatically for Material components and, for custom ones, via `minimumInteractiveComponentSize()`:

```
IconButton(
    onClick = onToggleBookmark,
    modifier = Modifier
        .minimumInteractiveComponentSize()   // expands hit area to 48dp, no visual change
        .clearAndSetSemantics {}             // (see advanced section)
) { Icon(Icons.Filled.BookmarkBorder, contentDescription = null) }
```

Listing 4: A 24 dp icon with a guaranteed 48 dp touch target.

This is the same pattern Gmail uses for the small "star" icon: a 24 dp glyph with a 48 dp hit area, so users do not open the email by accident when they meant to star it.

4.4 Scalable typography

Android 14 introduced **non-linear font scaling** up to 200%: small text grows a lot while already-large headings grow less, preserving hierarchy. To benefit, code must separate *text size* from *layout size*:

1. Text sizes always in `sp` — Compose applies the non-linear math automatically.
2. Paddings and constraints always in `dp`.

3. **Never** hard-code a `height()` on a text container; use `defaultMinSize` or padding so it can grow.

Figure 6 shows why the third rule matters: a fixed-height card clips its text at 200%, while the `sp`-based card grows gracefully. Our `Type.kt` defines every style in `sp`, so the app passes this test out of the box.

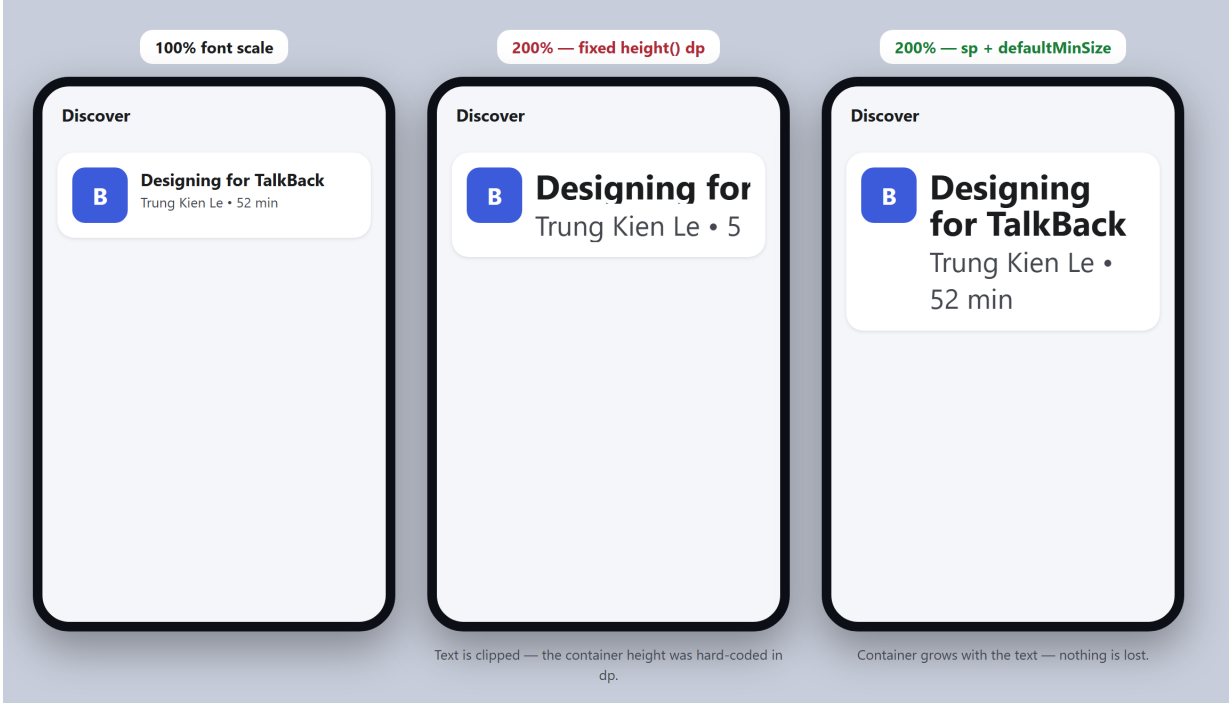


Figure 6: The same card at 100% and at 200% font scale. A hard-coded `height()` in dp (centre) clips the title; `sp` text with `defaultMinSize` (right) lets the container expand so nothing is lost.

4.5 Heading navigation

Finally, marking section titles with `semantics { heading() }` lets TalkBack users jump heading-to-heading instead of swiping through every item — the screen-reader equivalent of skimming. We apply it to the app-bar titles and the "Fresh for you" / "Episodes" headers.

5 Technical Approach II — Advanced Semantics & Navigation

Core semantics make a single component correct. Advanced semantics make a *screen* pleasant: they control how many focus stops there are, in what order they are read, and how deeply-nested actions are reached. These are the techniques that turn a technically-labelled UI into one that is actually comfortable to navigate by ear.

5.1 Semantic merging: fewer, richer focus stops

By default every text node is its own focus stop. A podcast card with a cover, title, author, and metadata therefore costs a TalkBack user 4–6 swipes to cross — multiplied across a list of shows, this is exhausting. `Modifier.semantics(mergeDescendants = true)` collapses a container's children into one node whose announcement is the concatenation of its parts.

```
Card(
```

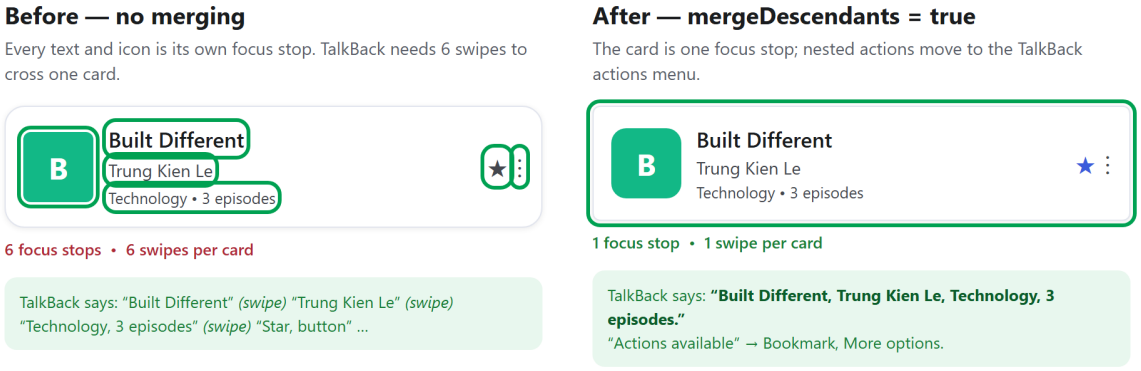


Figure 7: The effect of `mergeDescendants = true` on a podcast card. Before (left): six separate focus stops, six swipes. After (right): one focus stop that TalkBack reads as a single sentence, with the nested bookmark/menu actions moved into the actions menu.

```
    onClick = onOpen,
    modifier = Modifier.semantics(mergeDescendants = true) {
        customActions = listOf(
            CustomAccessibilityAction(bookmarkLabel) { onToggleBookmark(latest); true }
        )
    }
) { /* cover + title + author + metadata + nested icon buttons */ }
```

Listing 5: `PodcastCard.kt` — the card is one focus stop.

Constraint. A parent cannot merge a child that already merges itself — interactive components (`Button` , `Switch` , anything with `clickable` / `toggleable`) resist merging so they remain independently operable. This is exactly why the nested bookmark button needs the next technique.

5.2 Clearing and hiding: `clearAndSetSemantics` & `hideFromAccessibility`

Two modifiers let us prune the tree:

- `clearAndSetSemantics {}` wipes a node's semantics (and its descendants') and lets us redefine them — or leave them empty to remove the node from traversal entirely.
- `hideFromAccessibility()` (Compose 1.8+) hides a node from assistive services while keeping it for UI tests — the correct modern replacement for setting `contentDescription = null` on decorative elements.

In `CoverArt.kt` the coloured cover tile is decorative when a text title sits beside it, so we hide it. In `EpisodeRow.kt` the three trailing icon buttons are cleared so they do not each steal focus — their actions are re-exposed on the row (next subsection).

5.3 Custom accessibility actions

Nested icon buttons are an anti-pattern for screen readers: they are tiny targets and they clutter the swipe path (50 cards × 3 buttons = 150 extra swipes). The fix is to *clear* the physical buttons and *elevate* their operations to `customActions` on the parent row, where TalkBack surfaces them through its actions menu ("Actions available").

```
Row(
    modifier = Modifier.semantics(mergeDescendants = true) {
        customActions = listOf(
            CustomAccessibilityAction(playLabel)     { onPlay(); true },
            CustomAccessibilityAction(bookmarkLabel) { onToggleBookmark(); true },
            CustomAccessibilityAction(downloadLabel) { onToggleDownload(); true },
        )
    }
) {
    FilledIconButton(onClick = onPlay,
        modifier = Modifier.size(48.dp).clearAndSetSemantics {}) { /* icon */ }
    // ... title + metadata ...
    IconButton(onClick = onToggleBookmark,
        modifier = Modifier.minimumInteractiveComponentSize().clearAndSetSemantics {}) { /* icon */ }
}
```

Listing 6: `EpisodeRow.kt` — nested actions promoted to the row's actions menu.

A screen-reader user now hears the episode as one sentence, then opens "Actions" to Play, Bookmark, or Download — while sighted users still tap the visible 48 dp buttons directly.

5.4 Traversal order

Assistive tech reads geometrically (top-to-bottom, left-to-right), which breaks on non-linear layouts. Two modifiers fix it: `isTraversalGroup = true` draws a boundary that must be read completely before moving on, and `traversalIndex` (a `Float`, lowest first) reorders within that boundary. A common pitfall is setting `traversalIndex` *without* a group: because all nodes default to `0f` globally, the element is compared against the whole screen and jumps to the bottom. On the Discover screen we make the "Continue listening" banner read *first* even though it is not the top-most focusable item:

```
LazyColumn(modifier = Modifier.semantics { isTraversalGroup = true }) {
    item {
        ContinueListeningBanner(
            modifier = Modifier.semantics { traversalIndex = -1f } // read before everything else
        )
    }
    // ... section header + podcast cards ...
}
```

Listing 7: `HomeScreen.kt` — resume banner announced first via a low traversal index.

5.5 Decision matrix

Table 2 summarises when to reach for each advanced modifier.

6 Testing & Evaluation

Accessibility cannot be reduced to running one tool and reading a pass/fail number. Automated checkers, by W3C's own admission, catch only a fraction of real barriers; the rest needs human judgement and evidence from actual tasks. Our approach is therefore **evidence triangulation** across five layers (Figure 8), and — just as importantly — a discipline of separating *raw suggestions* from *confirmed defects*.

API	Applied to	Use case / effect
`mergeDescendants = true`	Container (Row/Column/Card)	Fuse fragmented text into one focus stop.
`clearAndSetSemantics {}`	Custom or nested nodes	Erase raw semantics; redefine or remove from traversal.
`hideFromAccessibility()`	Decorative elements	Hide from TalkBack, keep for UI tests.
`isTraversalGroup = true`	Layout container	Force children to be read as a bounded block.
`traversalIndex = Xf`	Child of a traversal group	Reorder reading (lowest value read first).
`customActions = listOf(...)`	Parent list item / card	Move nested actions into the TalkBack menu.

Table 2: Choosing an advanced semantics API by structural need.

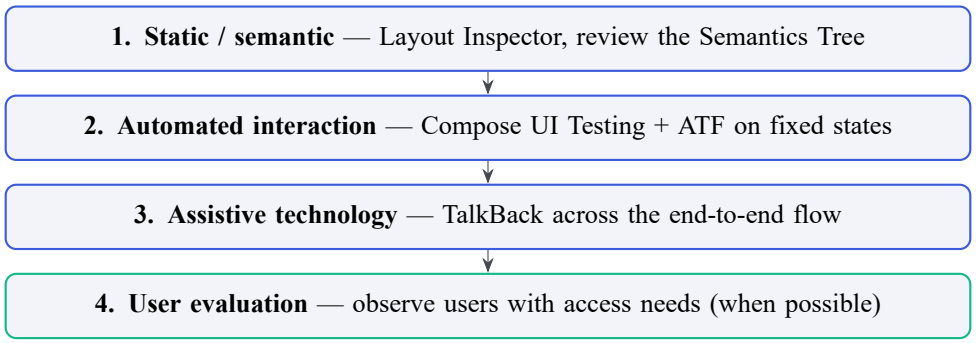


Figure 8: The layered evaluation model. Each layer answers a different question; no single layer is sufficient on its own.

6.1 Manual testing with TalkBack

TalkBack is the ground truth for the listening experience. Our protocol runs each core flow twice: first swiping linearly to check focus order and labels, then exploring by touch to compare the visual region with the focus region. For every important control we activate it and verify the role, state, and *post-action focus*. Common issues this surfaces — and which pure automation misses — are wrong or missing labels, focus that jumps after a dialog, over-merged cards that hide a child action, and dynamic changes (buffering, errors) that are never announced. We record each finding with a severity, reproduction steps, and expected-vs-actual behaviour.

6.2 Accessibility Scanner — and an honest case study

Accessibility Scanner evaluates a screen on-device and draws orange boxes around potential issues (labels, touch targets, contrast). To practise the *evidence-collection workflow* — not to judge another company's app — we captured a 15-screen flow through Agoda (Figure 9). The point of the exercise is methodological: how to record, de-duplicate, and interpret tool output responsibly.

The 15-screen capture produced **102 raw suggestions**, distributed as in Figure 10. These numbers must be read carefully: they are *not* 102 independent, confirmed bugs. Loading/skeleton states, web-view content, and the *same* chip appearing across several frames all inflate the count (report 2 and report 14 overlap; the splash screen produced none). The correct next step is de-duplication by root cause and confirmation with TalkBack — never turning a warning into a conclusion without reproducing it.

6.3 Layout Inspector, Compose UI Testing, and ATF

Once we have our own source, three developer tools close the loop:

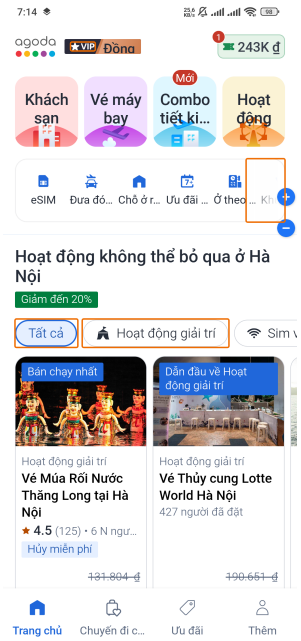


Figure 9: Accessibility Scanner on a real app (Agoda), used only to rehearse the workflow. Orange boxes mark raw *suggestions* — e.g. a 38 dp touch target below the 48 dp guideline and chips that may lack a distinct label. Raw suggestions are not confirmed defects.

- **Layout Inspector** shows the live merged and unmerged semantics trees, linking a TalkBack/Scanner symptom to a specific node — a diagnostic tool, not a verdict on user experience.
- **Compose UI Testing** asserts semantics contracts in CI: that a control has the accessible name a screen reader will speak, and `printToLog()` dumps the tree for diagnosis. Our `AccessibilityTest.kt` does exactly this.
- **Accessibility Test Framework (ATF)**, Google's open-source rule engine, adds automated checks for speakable text, touch-target size, and contrast, usable as a CI quality gate.

```
// 1. The icon-only search button must expose a spoken label, not just a glyph.
composeRule.onNodeWithContentDescription("Search podcasts").assertExists()

// 2. Dump the unmerged tree to verify merge/clear behaviour on the cards.
composeRule.onRoot(useUnmergedTree = true).printToLog("A11Y_TREE")
```

Listing 8: `AccessibilityTest.kt` — assert a spoken name, then dump the tree.

Table 3 positions each tool against the others.

Tool	Source?	Mode	Best for / cannot replace
TalkBack	No	Manual	Listening experience, focus, end-to-end flow.
Accessibility Scanner	No	Semi-auto	Fast visual triage of labels/targets/contrast; not coverage.
Layout Inspector	Debug build	Manual	Inspect merged/unmerged tree; not user experience.
Compose UI Testing	Yes	Automated	Semantics contracts & regression; not utterance quality.
ATF	Yes	Automated	Standardised checks in CI; not real usability.

Table 3: Complementary roles of the five verification tools.

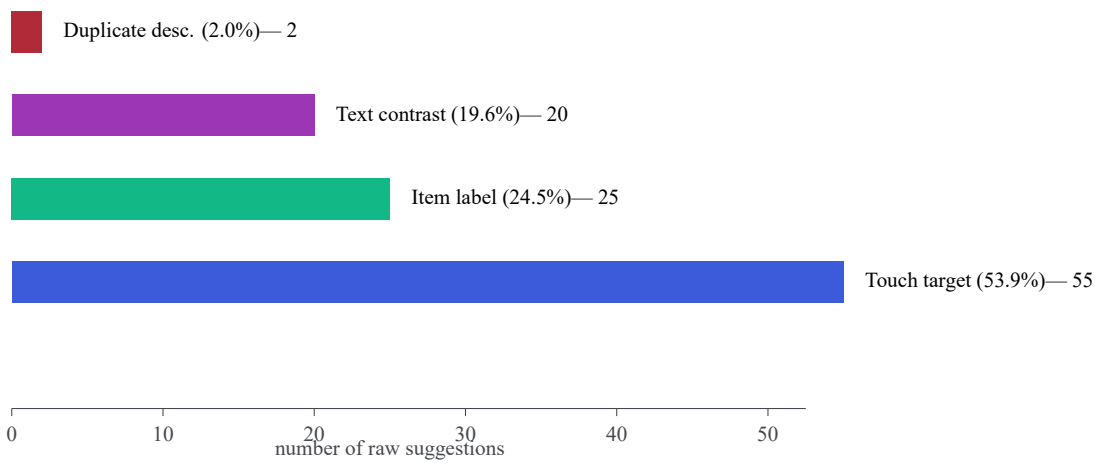


Figure 10: Distribution of the 102 raw suggestions across the 15-screen Agoda capture. Touch-target warnings dominate, but this is a *starting hypothesis*, not a root-cause finding.

6.4 Evaluation metrics and trade-offs

A credible evaluation states scope, representative flows, device, versions, and completion criteria, then reports both automated and task-based measures: first-flow coverage, task-completion rate, completion time, number of TalkBack gestures, automated pass rate, and a *confirmed-defect rate* that distinguishes real bugs from raw suggestions. Findings are triaged by **impact on the task** — *Blocker* (task impossible, fix before ship) > *Major* > *Moderate* > *Minor* — not merely by count. A missing label on the checkout button outweighs many touch-target warnings on secondary content.

Every technique in this report is a trade-off, and naming them is part of an honest evaluation:

- `mergeDescendants` cuts swipes but can hide an independent child action or produce an over-long sentence — measure gesture count *and* listen to the whole utterance.
- Rich labels aid comprehension but, if they duplicate role/state, add listening time.
- `traversalIndex` fixes reading order but hand-tuned indices are brittle and can drift from the visual layout; use the minimum number and document why.
- Custom controls give design freedom but must re-declare role, state, and keyboard behaviour that Material would have provided for free.

7 Conclusion

Accessibility in Jetpack Compose is not a checkbox but a design discipline that spans the whole stack. This project took an ordinary podcast UI and made it usable without sight by working the semantics tree deliberately at every level: naming and hiding elements correctly (Section 4), then shaping how those elements are grouped, ordered, and acted upon (Section 5). The result is a three-screen app in which a TalkBack user can discover a show, choose an episode, and control playback — hearing a clean sentence per card, reaching every action through the actions menu, and getting spoken feedback the moment playback state changes.

Equally important is the evaluation discipline of Section 6: TalkBack answers the listening question, Scanner triages quickly, Layout Inspector explains the structure, Compose UI Testing protects semantics against regressions, and ATF automates the mechanical checks. The value lies in *combining* these layers, defining a clear scope, and — as our Agoda case study showed — never mistaking a tool's raw suggestion for a confirmed defect.

Our broader takeaway is that accessible design is cheaper and better when it is built in from the first composable rather than retrofitted: preferring Material components with sound default semantics, reaching for custom semantics only where the design demands it, and testing with the same assistive technology real users rely on. Building inclusively is, in the end, simply building well.

Future work

The natural Version 2 is to run TalkBack on our own build across a device/version matrix, capture Layout Inspector evidence for representative findings, expand the Compose test suite and wire ATF into CI, and — resources permitting — invite users who rely on assistive technology to evaluate the core tasks.

Appendix on AI usage

AI assistants were used as a drafting and formatting aid: to help structure this report, generate the LaTeX scaffolding and diagrams, and produce the annotated UI mockups from our own designs and code. All accessibility techniques, the app architecture, the research decisions, and the case-study data originate from the group's own work; every AI-assisted passage was reviewed and edited by the members responsible for that section. Code samples in this report are excerpts from the app we wrote, not AI-generated boilerplate.

8 Appendix A — Running the app

1. Open the `PodcastAccessibilityApp` folder in Android Studio (Koala or newer); let it sync and generate the Gradle wrapper and `local.properties`.
2. Run the `app` configuration on an emulator or device (`minSdk 24`, `targetSdk 34`).
3. To experience the accessibility work, enable **TalkBack**, install **Accessibility Scanner**, and set the system **font size** to 200%.
4. Run the instrumented checks: `./gradlew connectedAndroidTest`.

Project layout. `data/` holds the models and the `PodcastRepository` interface with its mock; `ui/` holds the theme, navigation, and the `home`, `detail`, and `player` screens. Section 3, Table 1 maps every technique to its file.

Limitations of this report

This is a front-end study. The app ships with mocked data and no real audio engine, so performance and networked-state accessibility (e.g. announcing buffering under poor connectivity) are described at the design level rather than measured. The Agoda case study used a single device and one capture pass; its figures demonstrate the evidence-collection *process* and must not be read as an audit of Agoda. Absolute WCAG conformance is likewise not claimed — only a principled mapping of technology-neutral criteria onto Android.

References

[1] Android Developers. *Principles for improving app accessibility*. developer.android.com. Accessed 22 Jul 2026.

[2] Android Developers. *Semantics in Compose*. developer.android.com/develop/ui/compose/semantics.

[3] Google. *Get started on Android with TalkBack*. Android Accessibility Help.

[4] Android Developers. *Merging and clearing (Jetpack Compose accessibility)*. developer.android.com.

[5] Google. *Accessibility Scanner*. Google Play / Android Accessibility Help.

[6] Android Developers. *Accessibility Scanner — Accessibility on Android* (video).

[7] Android Developers. *Debug your layout with Layout Inspector* (Compose semantics view).

[8] Android Developers. *Accessibility in Compose — key concepts and the semantics tree*.

[9] Android Developers. *Test your Compose layout — merged and unmerged semantics trees*.

[10] Android Developers. *Modify traversal order* (`isTraversalGroup` , `traversalIndex`).

[11] Android Developers. *Testing your Compose layout — semantics matchers, assertions, printToLog*.

[12] Android Developers. *Automated accessibility checks in Compose tests* (ATF integration, Compose 1.8+).

[13] Google. *Accessibility Test Framework for Android* (open-source repository).

[14] Android Developers. *Accessibility checking with Espresso* (`AccessibilityChecks.enable()`).

[15] W3C Web Accessibility Initiative (WAI). *Selecting Web Accessibility Evaluation Tools* — automation covers only part of conformance.

[16] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 2023.

[17] W3C. *WCAG2ICT — Applying WCAG to Non-Web Information and Communications Technologies*.

[18] W3C WAI. *Website Accessibility Conformance Evaluation Methodology (WCAG-EM) 1.0*.

[19] W3C WAI. *Involving Users in Evaluating Web Accessibility*.

[20] ISO. *ISO 9241-11:2018 — Usability: definitions and concepts* (effectiveness, efficiency, satisfaction).

[21] W3C WAI. *Mobile Accessibility: How WCAG 2 and other WAI guidelines apply to mobile*.

[22] Google. *Material Design — Accessibility* (touch targets, contrast).

[23] European Commission. *European Accessibility Act (Directive (EU) 2019/882)*.

[24] Android Developers. *Android 14 features — non-linear font scaling*.

[25] Appt.org. *Accessibility action in Jetpack Compose*.